

1. Project Overview

Project Name: Permissions-RIT

One-line Pitch (What & Why):

Eg: A web app that helps to create permission letter for college purposes.

Problem Statement:

- Many students in my college when it comes to a case of having permission letter request letter doesn't know how to move with it, they go to some one and ask of making a letter for them, why not preset template for that
- The problem is faced by students of my college, when they ask the faculties for format it isn't that good
- They can use chatgpt to generate one i prefer a personalised one especially for rit kottayam

Target Users:

- Primary users: students
 - Secondary users: faculties
-

2. Goals & Success Criteria

Main Goal:

People using the software effectively for the permissions

Secondary Goals:

- People asking for more robust features
 -
-

3. User Stories / Use Cases

Write in simple language.

- As a **user**, I want to get the permission letter as pdf so that I can get it print and send them the content.
 - As an **admin**, I want to just manage template letters
-

4. Core Features (MVP First)

Must Have (MVP)

- Should able to download the letter as pdf
- Text formatting for a letter should be better
- Different template letters should be available

Nice to Have (Later)

- Shared editing on same letter
 - Direct mailing to the needed
-

5. Functional Requirements (What the system SHOULD do)

- FR1: The system shall have an option to write and format contents in a A4 size page
 - FR2: The system shall all the text editing features should be present there.
 - FR3: The system shall export the a4 letter as pdf
-

6. Non-Functional Requirements (How well it should work)

- Performance: main focus is for mobile phone so mobile oriented building
- Scalability: can hold up to many templates
- Security: not required initially

7. Tech Stack (Vibe-friendly)

Frontend: Next.js/React.js

Backend: if needed Node

Database: Fire base

AI / Automation (if any): not needed now

Hosting / Deployment: Vercel

APIs / Integrations: not needed

8. System Architecture (High Level)

Describe in words or draw later.

Flow:

User → Frontend → Backend → Processing → Storage → Response

Key Components:

- Frontend UI
 - Backend API
 - Image/Text Processing
 - Database
 - External services
-

9. Data Design

Entities / Tables / Collections:

10. Implementation Plan (Step-by-Step)

Phase 1 – Setup

- Repo setup
- Tech stack config
- Basic UI / API skeleton

Phase 2 – Core Logic

- Feature 1
- Feature 2
- Feature 3

Phase 3 – Integration

- Connect frontend ↔ backend
- Add storage / AI / automation

Phase 4 – Polish

- Error handling
- UI improvements
- Performance fixes